

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 3
з курсу “Інформаційна безпека програм та даних”
на тему: “Симетричні криптосистеми. Алгоритм RSA”

Виконав:

Студент. гр. ФЕП-41 Фіняк Олег

Перевірив:

Павлик М.Р.

Львів – 2026

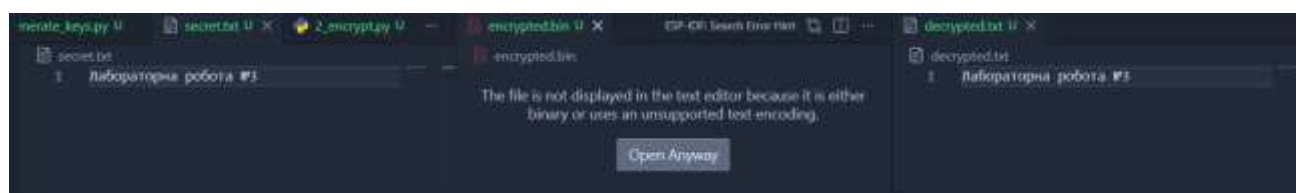
Завдання:

1. Встановіть крипто бібліотеку Python Pycryptodome: `python pip install pycryptodome`.
2. Запустіть код з прикладу та переконайтеся, що скрипт працює коректно.
3. Використовуючи даний приклад напишіть три скрипти:
 - перший скрипт генеруватиме закритий та відкритий ключі;
 - другий скрипт зашифруватиме текстовий файл за допомогою попередньо згенерованого відкритого ключа;
 - третій скрипт розшифруватиме зашифрований файл за допомогою попередньо згенерованого закритого ключа.

```
• Генерація пари RSA ключів (це може зайняти кілька секунд)...  
✓ Відкритий ключ збережено у файл 'public.pem'  
✓ Закритий ключ збережено у файл 'private.pem'
```

```
--- Шифрування RSA ---  
Введіть ім'я файлу для шифрування (напр., message.txt): secret.txt  
Введіть ім'я для зашифрованого файлу (напр., encrypted.bin): encrypted  
.bin  
✓ Файл успішно зашифровано! Результат у 'encrypted.bin'.
```

```
--- Розшифрування RSA ---  
Введіть ім'я зашифрованого файлу (напр., encrypted.bin): encrypted.bin  
  
Введіть ім'я для розшифрованого файлу (напр., decrypted.txt): decrypte  
d.txt  
✓ Файл успішно розшифровано! Результат у 'decrypted.txt'.
```



1_generate_keys.py

```
from Crypto.PublicKey import RSA #

def generate_keys():
    print("Генерація пари RSA ключів (це може зайняти кілька секунд)...")
    # Генерація ключів розміром 3072 біти
    keyPair = RSA.generate(3072) #

    # Створення та збереження відкритого ключа у файл
    pubKey = keyPair.publickey() #
    pubKeyPEM = pubKey.exportKey() #
    with open("public.pem", "wb") as file_out: #
        file_out.write(pubKeyPEM) #
    print("✓ Відкритий ключ збережено у файл 'public.pem'")

    # Створення та збереження закритого ключа у файл
    privKeyPEM = keyPair.exportKey() #
    with open("private.pem", "wb") as file_out: #[cite: 3]
        file_out.write(privKeyPEM) #[cite: 3]
    print("✓ Закритий ключ збережено у файл 'private.pem'")

if __name__ == "__main__":
    generate_keys()
```

2_encrypt.py

```
from Crypto.PublicKey import RSA #[cite: 3]
from Crypto.Cipher import PKCS1_OAEP #[cite: 3]

def encrypt_file(input_filename, output_filename):
    try:
        # Імпорт відкритого ключа з файлу
        publickey = RSA.importKey(open('public.pem').read()) #[cite: 3]
        encryptor = PKCS1_OAEP.new(publickey) #[cite: 3]

        # Зчитування тексту з файлу
        with open(input_filename, 'r', encoding='utf-8') as f:
            msg = f.read()

        # Шифрування повідомлення
        encrypted = encryptor.encrypt(msg.encode('utf-8')) #[cite: 3]

        # Збереження зашифрованих байтів у файл
        with open(output_filename, 'wb') as f:
            f.write(encrypted)

        print(f"✓ Файл успішно зашифровано! Результат у '{output_filename}'.")

    except FileNotFoundError:
        print("✗ Помилка: Не знайдено файл з текстом або файл 'public.pem'.")
```

```

if __name__ == "__main__":
    print("--- Шифрування RSA ---")
    in_file = input("Введіть ім'я файлу для шифрування (напр., message.txt): ")
    out_file = input("Введіть ім'я для зашифрованого файлу (напр., encrypted.bin): ")
    encrypt_file(in_file, out_file)

```

3_decrypt.py

```

from Crypto.PublicKey import RSA #[cite: 3]
from Crypto.Cipher import PKCS1_OAEP #[cite: 3]

def decrypt_file(input_filename, output_filename):
    try:
        # Імпорт закритого ключа з файлу
        privatekey = RSA.importKey(open('private.pem').read()) #[cite: 3]
        decryptor = PKCS1_OAEP.new(privatekey) #[cite: 3]

        # Зчитування зашифрованих даних
        with open(input_filename, 'rb') as f:
            encrypted = f.read()

        # Розшифрування повідомлення
        decrypted = decryptor.decrypt(encrypted) #[cite: 3]

        # Збереження розшифрованого тексту
        with open(output_filename, 'w', encoding='utf-8') as f:
            f.write(decrypted.decode('utf-8'))

        print(f"✓ Файл успішно розшифровано! Результат у '{output_filename}'.")

    except FileNotFoundError:
        print("✗ Помилка: Не знайдено зашифрований файл або файл 'private.pem'.")
    except ValueError:
        print("✗ Помилка: Не вдалося розшифрувати файл. Можливо, використано неправильний закритий ключ.")

if __name__ == "__main__":
    print("--- Розшифрування RSA ---")
    in_file = input("Введіть ім'я зашифрованого файлу (напр., encrypted.bin): ")
    out_file = input("Введіть ім'я для розшифрованого файлу (напр., decrypted.txt): ")
    decrypt_file(in_file, out_file)

```

Висновок: Під час виконання лабораторної роботи було успішно реалізовано алгоритм асиметричного шифрування RSA за допомогою бібліотеки Rucryptodome. На практиці розроблено та протестовано скрипти для генерації пари ключів, шифрування файлу відкритим ключем та його подальшого розшифрування закритим ключем, що наочно демонструє механізм захисту даних в асиметричних криптосистемах.